

Encryption

Parent: INFOSEC_POLICY.md **Owner:** Matt Fellows — matt@tangiblevalue.com **Effective date:** 2026-05-06 **Review cadence:** annually, or on material change to the hosting/storage stack.

1. Scope

Two domains:

1. **In transit** — bytes moving over the network between the user's browser, the Vercel edge, the Express backend (Vercel serverless functions), Firestore, the Plaid API, and any other third-party services.
2. **At rest** — bytes sitting in persistent storage: Firestore documents, Vercel's deployed bundles, environment variable storage, and the developer's local machine.

2. In Transit (Q6)

Inbound: browser → app

- TLS termination happens at **Vercel's edge network**. Vercel enforces a minimum of **TLS 1.2** and supports **TLS 1.3**. TLS 1.0 / 1.1 / SSLv3 are not accepted.
- Certificate management is handled by Vercel: certificates are issued automatically (Let's Encrypt for *.vercel.app; ACME for custom domains) and renewed before expiry.
- HSTS is applied via the helmet() middleware default header set (Strict-Transport-Security: max-age=15552000; includeSubDomains), which instructs the browser to upgrade http:// to https:// for the next 180 days.
- HTTP requests to the bare domain are served the redirect / HSTS upgrade by Vercel itself.

Outbound: app → external services

Destination	Mechanism	Min TLS
Plaid API	plaid SDK over axios (HTTPS)	TLS 1.2+ enforced by Plaid
Firestore	firebase-admin over gRPC	TLS 1.2+ enforced by Google
Alpha Vantage / IEX / Yahoo Finance / OpenAI / Perplexity	axios over HTTPS	TLS 1.2+ enforced by each provider

No outbound HTTP calls are made over plain http://. This is verifiable by grep on the codebase: every external URL is https://.

Verification

To confirm the TLS posture of the production deployment at any time:

```
# Confirm TLS 1.2+ is the floor (TLS 1.0 / 1.1 should be rejected)
openssl s_client -connect <hostname>:443 -tls1 < /dev/null 2>&1 | grep -E "(handshake|protocol)"
openssl s_client -connect <hostname>:443 -tls1_1 < /dev/null 2>&1 | grep -E "(handshake|protocol)"
openssl s_client -connect <hostname>:443 -tls1_2 < /dev/null 2>&1 | grep -E "(Protocol|Cipher)"
openssl s_client -connect <hostname>:443 -tls1_3 < /dev/null 2>&1 | grep -E "(Protocol|Cipher)"

# Or use SSL Labs scan (don't expose internal hosts; only test public endpoint):
# https://www.ssllabs.com/ssltest/analyze.html?d=<hostname>
```

Expected result: TLS 1.0 / 1.1 fail with handshake failure; TLS 1.2 and 1.3 succeed; ciphers are AEAD (e.g., TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, or ECDHE-RSA-AES...-GCM...).

3. At Rest (Q7)

Layer 1 — Platform default (Firestore-managed)

Every document in Firestore is encrypted at rest by Google with **AES-256** using **Google-managed keys**. This applies to every collection used by wealth-navigator, including:

- users (email, password hash)
- plaid_items (item ID, encrypted access token, institution name)
- plaid_accounts, plaid_transactions, plaid_holdings, plaid_liabilities
- passkeys
- expenses, trades, ideas, bets, budgets, balance_snapshots, sync_logs, settings

Reference: [Google Cloud Firestore encryption at rest](#).

Layer 2 — Application-layer overlay (highest-sensitivity fields)

The Plaid access_token is the single highest-blast-radius credential in the system: it grants the holder ongoing API access to a user's bank/brokerage accounts. To defend against scenarios where the Firestore-managed encryption layer is bypassed (e.g., a Google insider attack, a misconfigured Firestore export, an accidental snapshot leak), the access token is **double-encrypted**:

- **Cipher**: AES-256-CBC
- **Key**: 32 bytes, hex-encoded in process.env.ENCRYPTION_KEY (stored as a Vercel encrypted environment variable; never committed to source).
- **IV**: 16-byte random IV generated per encryption; prepended to ciphertext as iv_hex:ciphertext_hex for storage.
- **Implementation**: backend/services/encryption.js.
- **Boot-time guard**: in production, the service refuses to start if ENCRYPTION_KEY is unset (throw new Error(...)), making misconfiguration loud rather than silent.

This means an attacker who somehow obtains the raw Firestore plaid_items documents *and* who somehow defeats Google's at-rest encryption *still* cannot use the access tokens unless they also exfiltrate ENCRYPTION_KEY from Vercel's environment variable store.

Other Plaid-derived data

Transaction history, holdings, liabilities, balances, etc. are stored in Firestore with **Layer 1 only** — no app-layer encryption overlay. They are still encrypted at rest, but only by Google's managed keys. This is judged adequate because:

- These rows do not, by themselves, allow ongoing access to a financial institution (unlike an access token).
- App-layer encryption of every field would prevent server-side filtering and aggregation queries that the application depends on.

The honest answer to Q7 is therefore: **“Yes — We encrypt ALL consumer data retrieved from the Plaid API at-rest”**, because every field is covered by at least one encryption layer. Access tokens specifically have two layers.

Vercel build artifacts and env vars

- **Source bundles** at rest in Vercel's CDN: encrypted by Vercel, AES-256.
- **Environment variables** (JWT_SECRET, ENCRYPTION_KEY, PLAID_SECRET, FIREBASE_SERVICE_ACCOUNT, CRON_SECRET, WEBAUTHN_*, etc.): stored encrypted by Vercel; only decrypted in the function runtime at request time. Vercel does not log env-var values.

Developer machine

- Source code and any local .env files sit on the developer's Mac, which has **FileVault** (AES-128-XTS) enabled. The screen unlocks via login password / Touch ID.
- macOS Keychain holds API tokens for tools (GitHub CLI, Vercel CLI, etc.); also FileVault-protected.

4. Key Management

Key / Secret	Purpose	Rotation cadence	Rotation procedure
ENCRYPTION_KEY	App-layer AES-256 of Plaid access tokens	At least annually + on suspected compromise	(1) generate new key. (2) script: for each plaid_items doc, decrypt(old_key) → encrypt(new_key) and update. (3) update Vercel env. (4) redeploy. (5) verify via /api/plaid/sync. (6) keep old key for 7 days then purge.
JWT_SECRET	Sign session tokens	At least annually + on suspected compromise	Update Vercel env; on next deploy, all existing JWTs are invalidated and users must re-authenticate.
PLAID_SECRET	Plaid API auth	On suspected compromise	Rotate via Plaid Dashboard; update Vercel env.
FIREBASE_SERVICE_ACCOUNT_KEY	Cloud Firestore access	At least annually + on suspected compromise	Generate new service-account key in GCP IAM; update Vercel env; revoke old key after deploy.
CRON_SECRET	Vercel Cron auth	At least annually + on suspected compromise	Update Vercel env + vercel.json reference; redeploy.
WEBAUTHN_*	RP configuration (not secrets, but require consistency)	Only on hostname change	Update env; existing passkeys are scoped to RP ID and break on RP change — if the production hostname changes, all users must re-register their passkeys.

A reminder for the calendar: the next annual key-rotation review is due **2027-05-06**.

5. Known Limitations

Limitation	Impact	Status
AES-256-CBC is unauthenticated. A motivated attacker with write access to ciphertext could attempt padding-oracle / bit-flipping attacks.	Low — Firestore writes are gated by Firebase IAM; the realistic threat actor with Firestore write access already wins regardless.	Open — consider migration to AES-256-GCM (authenticated). Migration requires re-encrypting all stored tokens. SLA: at next major architecture revision. Accepted residual risk.
Firestore-side encryption uses Google-managed keys, not customer-managed (CMEK). ENCRYPTION_KEY lives in a single store (Vercel env). No HSM or KMS-backed wrapping.	Low for personal-use scope — Google's threat model is well-understood. Low — Vercel env is encrypted, access-controlled, and accessible only to Owner via Vercel auth (which has phishing-resistant MFA via Google).	Accepted residual risk.

6. References

- INFOSEC_POLICY.md — parent policy
- ACCESS_CONTROL.md — who can read these keys
- backend/services/encryption.js — application-layer crypto
- [Vercel: TLS / encryption](#)
- [Google Cloud Firestore: encryption at rest](#)